

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5117 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

I CH.Bhavya Sri/192411249(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

CH.Bhavya Sri

Signature of the student:

***Annexure A Coding
Challenge***

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

1. IPSec Authentication Header (AH) – MAC Generation and Verification

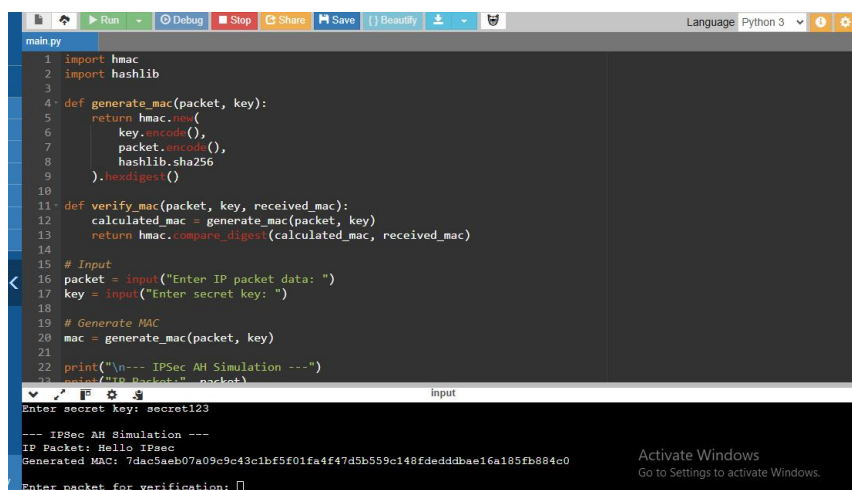
Aim

To simulate the IPSec AH process by generating and verifying a MAC for IP packet integrity.

Algorithm

1. Start the program.
2. Read the **IP packet data** from the user.
3. Read the **secret key**.
4. Apply a MAC algorithm such as **HMAC-SHA256** to the packet using the secret key.
5. Store the generated MAC.
6. Display the generated MAC.
7. Read the packet received for verification.
8. Generate a new MAC for the received packet using the same key.
9. Compare the newly generated MAC with the original MAC.
10. If both MACs are equal, display "**MAC Verification Successful**".
11. Otherwise, display "**MAC Verification Failed**".
12. Stop.

OUTPUT



```
main.py
1 import hmac
2 import hashlib
3
4 def generate_mac(packet, key):
5     return hmac.new(
6         key.encode(),
7         packet.encode(),
8         hashlib.sha256
9     ).hexdigest()
10
11 def verify_mac(packet, key, received_mac):
12     calculated_mac = generate_mac(packet, key)
13     return hmac.compare_digest(calculated_mac, received_mac)
14
15 # Input
16 packet = input("Enter IP packet data: ")
17 key = input("Enter secret key: ")
18
19 # Generate MAC
20 mac = generate_mac(packet, key)
21
22 print("\n--- IPSec AH Simulation ---")
23 print("Generated MAC: " + mac)
24
25 # Verification
26 received_packet = input("Enter packet for verification: ")
27
28 # Verify
29 if verify_mac(received_packet, key, mac):
30     print("MAC Verification Successful")
31 else:
32     print("MAC Verification Failed")
```

Input

Enter secret key: secret123

--- IPSec AH Simulation ---

IP Packet: Hello IPsec

Generated MAC: 7dac5aeb07a09c9c43c1bf5f01fa4f47d5b559c148fdedddbae16a185fb884c0

Enter packet for verification:

Explanation

The program uses **HMAC-SHA256** to generate a MAC from the IP packet and a secret key. When the packet is received, the MAC is calculated again and compared with the original MAC. If both match, the packet has not been modified.

2. IPsec ESP – Encryption and Decryption

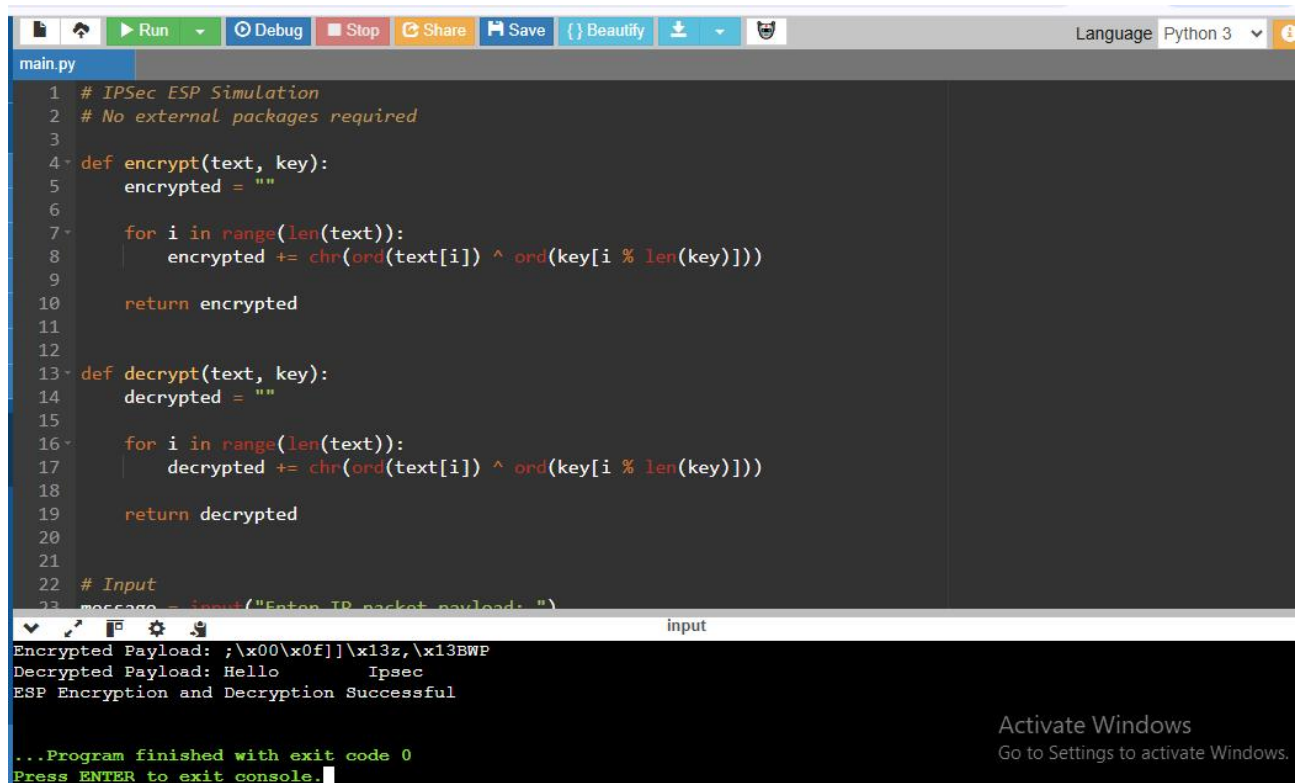
Aim

To implement a simplified ESP process for encrypting and decrypting an IP packet payload.

Algorithm

1. Start the program.
2. Read the **IP packet payload**.
3. Read the **secret key**.
4. Convert each character of the payload into its numerical value.
5. Perform the XOR operation between the payload character and the corresponding key character.
6. Store the resulting values as the **encrypted payload**.
7. Display the encrypted payload.
8. Perform the XOR operation again using the same key.
9. Obtain the **decrypted payload**.
10. Display the decrypted payload.
11. Compare the decrypted payload with the original payload.
12. If both are equal, display "**ESP Encryption and Decryption Successful**".
13. Otherwise, display "**Decryption Failed**".
14. Stop.

OUTPUT:



The screenshot shows a Python IDE with a file named `main.py`. The code implements a simple XOR-based encryption and decryption simulation for IPsec ESP. It includes comments indicating that no external packages are required. The `encrypt` function takes a text string and a key, and the `decrypt` function takes the encrypted text and the same key. Both functions use a loop to perform a bitwise XOR operation on each character of the text with the corresponding character of the key. The program prompts the user to enter an IP packet payload, which is then encrypted and displayed as a hexadecimal string. The encrypted payload is then decrypted back to the original message, "Hello Ipsec", and a success message is printed. The program ends with an exit code of 0.

```
1 # IPsec ESP Simulation
2 # No external packages required
3
4 def encrypt(text, key):
5     encrypted = ""
6
7     for i in range(len(text)):
8         encrypted += chr(ord(text[i]) ^ ord(key[i % len(key)]))
9
10    return encrypted
11
12
13 def decrypt(text, key):
14     decrypted = ""
15
16     for i in range(len(text)):
17         decrypted += chr(ord(text[i]) ^ ord(key[i % len(key)]))
18
19    return decrypted
20
21
22 # Input
23 message = input("Enter IP packet payload: ")
24
25 input
26
27 Encrypted Payload: ;\x00\x0f]\x13z,\x13BWP
28 Decrypted Payload: Hello      Ipsec
29 ESP Encryption and Decryption Successful
30
31 ...Program finished with exit code 0
32 Press ENTER to exit console.
```

Explanation

The program simulates the basic idea of **ESP encryption and decryption**.

- The user enters an IP packet payload.
- A secret key is entered.
- The payload is encrypted using a simple XOR operation.
- The encrypted payload is decrypted using the same key.
- If the decrypted message matches the original message, the process is successful.

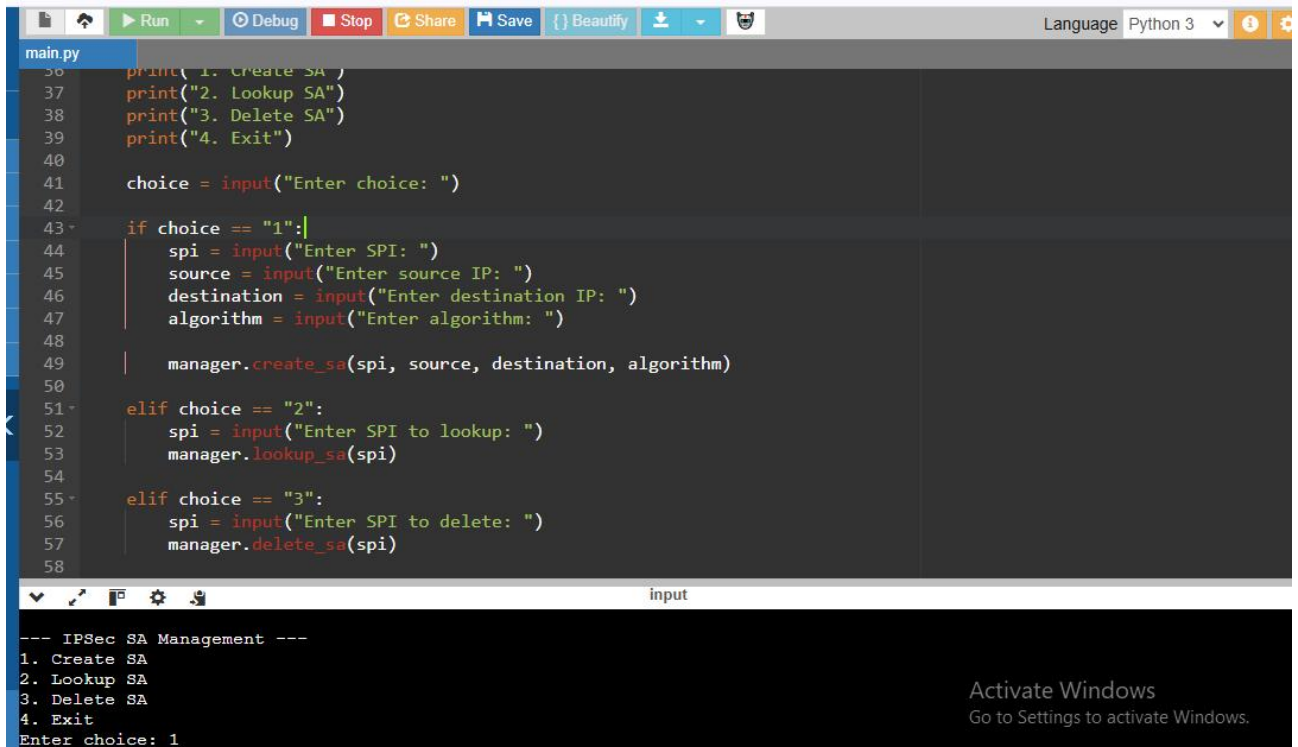
Note: XOR here is only for educational simulation. It is **not actual ESP/AES encryption**.

3. IPSec Security Association (SA) Management

Aim

To create a program that manages Security Associations by performing **creation, lookup, and deletion** operations.

OUTPUT:



```
main.py
36 print("1. Create SA")
37 print("2. Lookup SA")
38 print("3. Delete SA")
39 print("4. Exit")
40
41 choice = input("Enter choice: ")
42
43 if choice == "1":
44     spi = input("Enter SPI: ")
45     source = input("Enter source IP: ")
46     destination = input("Enter destination IP: ")
47     algorithm = input("Enter algorithm: ")
48
49     manager.create_sa(spi, source, destination, algorithm)
50
51 elif choice == "2":
52     spi = input("Enter SPI to lookup: ")
53     manager.lookup_sa(spi)
54
55 elif choice == "3":
56     spi = input("Enter SPI to delete: ")
57     manager.delete_sa(spi)
58
input
--- IPSec SA Management ---
1. Create SA
2. Lookup SA
3. Delete SA
4. Exit
Enter choice: 1
```

Explanation

A **Security Association (SA)** stores parameters required for IPSec communication. The program uses the **SPI (Security Parameters Index)** as a unique identifier and supports creating, searching, and deleting SAs.

4. Simplified PGP for Secure Email

Aim

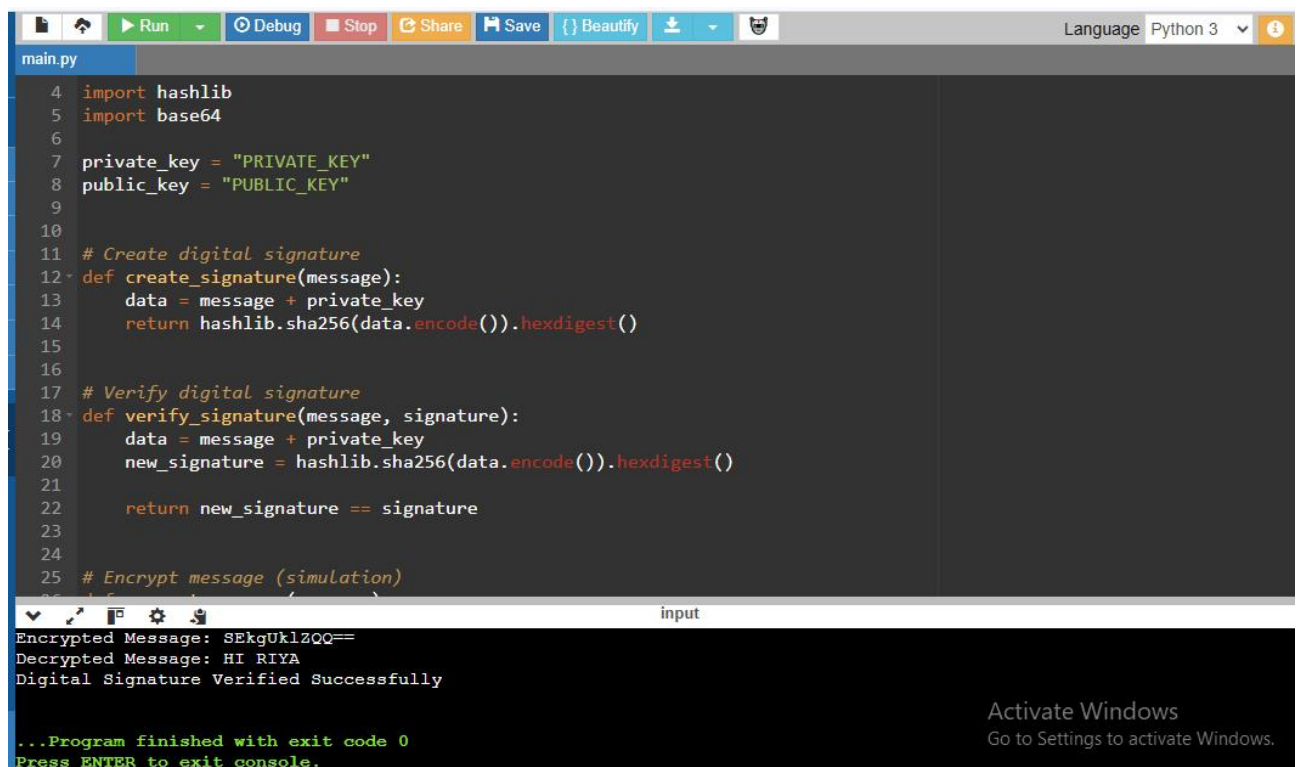
To implement a simplified PGP system using **public-key encryption and digital signatures**.

Algorithm

1. Start the program.
2. Define a private key and public key for the simulation.
3. Read the **email message** from the user.

4. Append the private key to the message.
5. Apply **SHA-256 hashing** to generate a digital signature.
6. Store the generated signature.
7. Encode the email message using Base64 to simulate encryption.
8. Display the encrypted message and digital signature.
9. Decode the encrypted message to obtain the original message.
10. Generate a new signature for the decrypted message.
11. Compare the new signature with the original signature.
12. If both signatures match, display "**Digital Signature Verified Successfully**".
13. Otherwise, display "**Digital Signature Verification Failed**".
14. Stop.

OUTPUT



The screenshot shows a Python IDE with a file named `main.py`. The code implements a PGP-like secure email system. It includes functions for creating a digital signature, verifying a digital signature, and simulating message encryption. The output of the program is displayed in the console window at the bottom.

```
4 import hashlib
5 import base64
6
7 private_key = "PRIVATE_KEY"
8 public_key = "PUBLIC_KEY"
9
10
11 # Create digital signature
12 def create_signature(message):
13     data = message + private_key
14     return hashlib.sha256(data.encode()).hexdigest()
15
16
17 # Verify digital signature
18 def verify_signature(message, signature):
19     data = message + private_key
20     new_signature = hashlib.sha256(data.encode()).hexdigest()
21
22     return new_signature == signature
23
24
25 # Encrypt message (simulation)
```

Input

```
Encrypted Message: SEkgUklZQQ==
Decrypted Message: HI RIYA
Digital Signature Verified Successfully

...Program finished with exit code 0
Press ENTER to exit console.
```

Activate Windows
Go to Settings to activate Windows.

Explanation

The program demonstrates the basic operations of a **PGP-like secure email system**:

1. The user enters an email message.
2. A hash-based digital signature is generated.
3. The message is encoded to simulate encryption.
4. The receiver decodes the message.
5. The digital signature is verified.
6. If the signature matches, the message is considered authentic and unchanged

5. Email Spam Classification

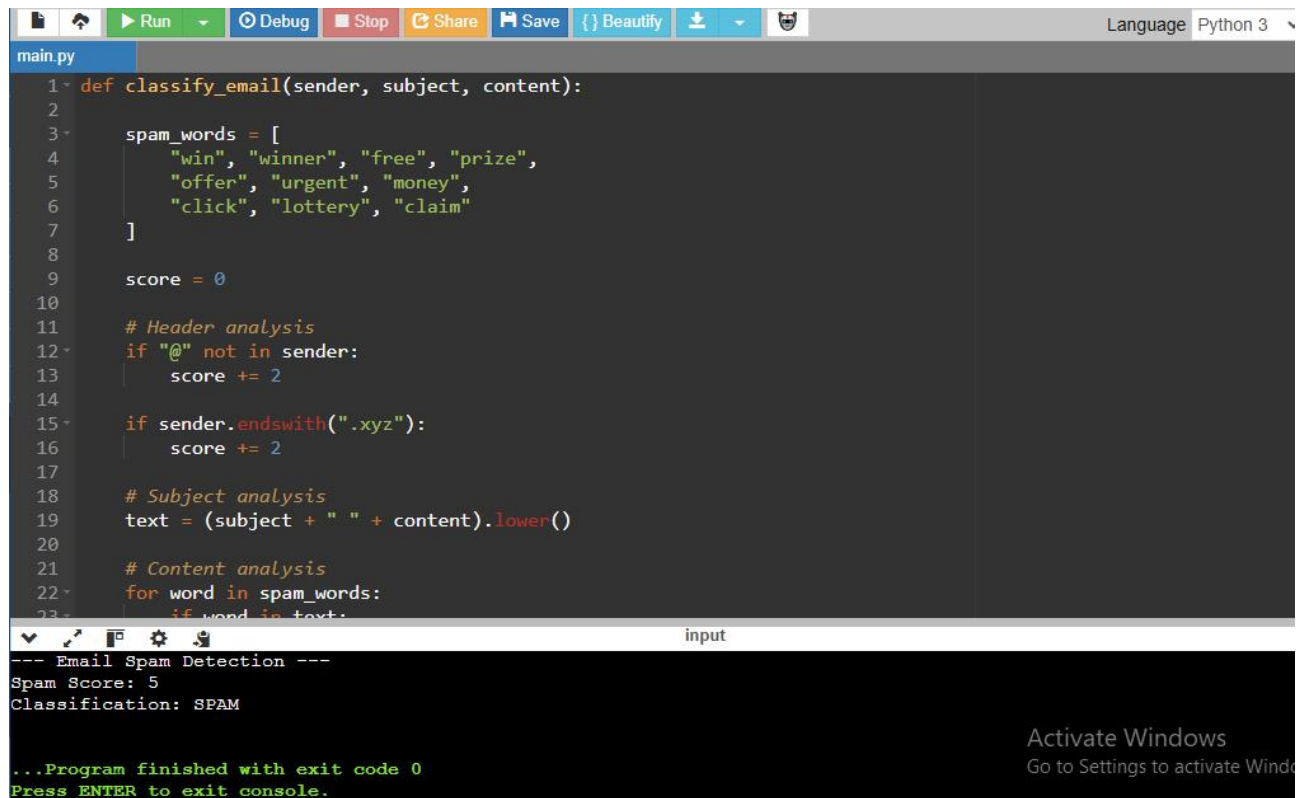
Aim

To classify emails as **spam** or **legitimate** using header analysis and content-based filtering

Algorithm

1. Start the program.
2. Read the **sender email address**.
3. Read the **email subject**.
4. Read the **email content**.
5. Initialize the spam score to **0**.
6. Check the sender address.
7. If the sender contains suspicious information, increase the spam score.
8. Check the email subject and content for common spam words such as:
 1. Free
 2. Prize
 3. Winner
 4. Urgent
 5. Money
 6. Click
 7. Lottery
9. Increase the spam score for each suspicious word found.
10. Compare the final score with a predefined threshold.
11. If the score is greater than or equal to the threshold, classify the email as **SPAM**.
12. Otherwise, classify it as **LEGITIMATE**.
13. Display the spam score and classification.
14. Stop.

OUTPUT:



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'classify_email' that takes 'sender', 'subject', and 'content' as arguments. It initializes a list of spam words: 'win', 'winner', 'free', 'prize', 'offer', 'urgent', 'money', 'click', 'lottery', and 'claim'. A 'score' variable is set to 0. The function performs three analyses: 1. Header analysis: checks if '@' is not in the sender (adds 2 to score) and if the sender ends with '.xyz' (adds 2 to score). 2. Subject analysis: concatenates the subject and content, converts to lowercase, and checks for spam words. 3. Content analysis: checks for spam words in the content. The output console shows the results of running the program: '--- Email Spam Detection ---', 'Spam Score: 5', 'Classification: SPAM', and a message indicating the program finished with exit code 0.

```
1 def classify_email(sender, subject, content):
2
3     spam_words = [
4         "win", "winner", "free", "prize",
5         "offer", "urgent", "money",
6         "click", "lottery", "claim"
7     ]
8
9     score = 0
10
11     # Header analysis
12     if "@" not in sender:
13         score += 2
14
15     if sender.endswith(".xyz"):
16         score += 2
17
18     # Subject analysis
19     text = (subject + " " + content).lower()
20
21     # Content analysis
22     for word in spam_words:
23         if word in text:
```

input

--- Email Spam Detection ---
Spam Score: 5
Classification: SPAM

...Program finished with exit code 0
Press ENTER to exit console.

Explanation

The program checks both **email headers and content**. It looks for suspicious sender information and common spam words such as *free*, *prize*, *urgent*, *click*, and *lottery*. A score is calculated, and the email is classified based on a threshold.

This is a **rule-based spam detection system**. It is simple and suitable for demonstrating the basic concept in a lab.